# BÀI THỰC HÀNH 2

Lê Văn Triệu – M2526034
Học phần CT999 – Công nghệ số nâng cao

## Bài tập 1 – Thực hành Data Wrangling với Python/Pandas

Chọn dữ liệu taxi từ kho dữ liệu mẫu của Seaborn để thực hành làm sạch, chuẩn hóa ngày và tạo cột tính toán. Nguồn: https://github.com/mwaskom/seaborn-data/blob/master/taxis.csv

### Bước 1. Đọc và xem dữ liệu gốc

Dùng pd.read_csv() đọc du_lieu/taxis_goc.csv, sau đó xem df.shape, df.columns và df.head(). Dữ liệu có 6.433 dòng và 14 cột. Các cột thể hiện thời điểm đón/trả, hành khách, quãng đường, các khoản tiền, màu xe, thanh toán và khu vực đón/trả. Hình dưới liệt kê đầy đủ tên cột và một số dòng đầu.

Dùng lệnh để xem dữ liệu:

```
(.venv) C:\lvtrieu\ths\CT999-Cong-nghe-so-nang-cao\bai-th-2\bai-1-2>python
Python 3.13.17 (tags/v3.13.17:64e080d, Oct 21 2026, 14:08:53) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import pandas as pd
...
... df = pd.read_csv("du_lieu/taxis_goc.csv")
...
... print("So dong:", df.shape[0])
... print("So cot:", df.shape[1])
...
... print("\nDanh sach cot:")
... print(df.columns.tolist())
...
... print("\n5 dong dau:")
... print(df.head().to_string(index=False))
...
```

```
So dong: 6433
So cot: 14

Danh sach cot:
['pickup', 'dropoff', 'passengers', 'distance', 'fare', 'tip', 'tolls', 'total', 'color', 'payment', 'pickup_zone', 'dro
poff_zone', 'pickup_borough', 'dropoff_borough']

5 dong dau:
            pickup             dropoff  passengers  distance  fare  tip  tolls  total  color     payment           pick
up_zone          dropoff_zone pickup_borough dropoff_borough
2019-03-23 20:21:09 2019-03-23 20:27:24           1      1.60   7.0 2.15    0.0  12.95 yellow credit card       Lenox Hi
ll West   UN/Turtle Bay South      Manhattan       Manhattan
2019-03-04 16:11:55 2019-03-04 16:19:00           1      0.79   5.0 0.00    0.0   9.30 yellow        cash Upper West Sid
e South Upper West Side South      Manhattan       Manhattan
2019-03-27 17:53:01 2019-03-27 18:00:25           1      1.37   7.5 2.36    0.0  14.16 yellow credit card         Alphab
et City          West Village      Manhattan       Manhattan
2019-03-10 01:23:59 2019-03-10 01:49:51           1      7.70  27.0 6.15    0.0  36.95 yellow credit card             Hu
dson Sq        Yorkville West      Manhattan       Manhattan
2019-03-30 13:27:42 2019-03-30 13:37:14           3      2.16   9.0 1.10    0.0  13.40 yellow credit card          Midto
wn East        Yorkville West      Manhattan       Manhattan
>>>
```

*Hình 1. Đọc dữ liệu CSV, kiểm tra số dòng, số cột, tên cột và năm dòng đầu tiên.*

Các hình trong báo cáo là ảnh chụp lệnh thực hiện và kết quả chạy Python/Pandas trên CMD.

*Bộ minh chứng là thư mục m2526034-le-van-trieu đặt cùng cấp với báo cáo. Các đường dẫn du_lieu/... và ket_qua/... được tính bên trong thư mục này.*

**Bước 2. Loại bỏ dòng có giá trị thiếu**

Dùng df.isna().any(axis=1) để tìm dòng có ít nhất một ô thiếu, sau đó df.dropna() để loại bỏ. Có 92 dòng như vậy. Ví dụ, chỉ số dòng 7 thiếu payment, dòng 42 thiếu pickup_zone và dropoff_zone. Một dòng có nhiều ô thiếu chỉ được tính một lần. Sau bước này còn 6.341 dòng.

```
>>> df = pd.read_csv("du_lieu/taxis_goc.csv")
...
... print("So dong ban dau:", len(df))
... print("So dong co o thieu:", df.isna().any(axis=1).sum())
...
... print(df.loc[
...     df.isna().any(axis=1),
...     ["payment", "pickup_zone", "dropoff_zone"]
... ].head())
...
... df = df.dropna().copy()
...
... print("So dong con lai:", len(df))
... print("So o thieu con lai:", df.isna().sum().sum())
...
```

```
...
So dong ban dau: 6433
So dong co o thieu: 92
         payment        pickup_zone     dropoff_zone
7            NaN        Murray Hill         Flatiron
42   credit card                NaN              NaN
445          NaN        Boerum Hill  Columbia Street
491          NaN        Murray Hill      Murray Hill
545          NaN  LaGuardia Airport     Forest Hills
So dong con lai: 6341
So o thieu con lai: 0
>>>
```

*Hình 2. Kiểm tra và loại bỏ dòng có giá trị thiếu bằng dropna().*

**Bước 3. Kiểm tra và loại dòng trùng**

Dùng df.duplicated().sum() để đếm và df.drop_duplicates() để loại dòng trùng hoàn toàn. Kết quả đếm bằng 0 nên số dòng vẫn là 6.341.

```
>>> print("So dong truoc khi kiem tra:", len(df))
... print("So dong trung:", df.duplicated().sum())
...
... df = df.drop_duplicates()
...
... print("So dong sau khi loai trung:", len(df))
...
```

```
...
So dong truoc khi kiem tra: 6341
So dong trung: 0
So dong sau khi loai trung: 6341
```

*Hình 3. Kiểm tra và loại dòng trùng hoàn toàn bằng drop_duplicates().*

**Bước 4. Chuyển kiểu ngày giờ**

Dùng pd.to_datetime() cho pickup và dropoff để có thể tính chênh lệch thời gian. Kiểm tra trên dữ liệu sau khi loại dòng thiếu không phát hiện giá trị ngày giờ không chuyển đổi được. Vì vậy đây là bước chuyển kiểu dữ liệu, không phải bằng chứng ngày giờ gốc bị sai.

```
>>> print("Kieu du lieu truoc khi chuyen:")
... print(df[["pickup", "dropoff"]].dtypes)
...
... pickup_dt = pd.to_datetime(df["pickup"], errors="coerce")
... dropoff_dt = pd.to_datetime(df["dropoff"], errors="coerce")
...
... print("\nKieu du lieu sau khi chuyen:")
... print("pickup:", pickup_dt.dtype)
... print("dropoff:", dropoff_dt.dtype)
...
... print("\nSo gia tri khong chuyen doi duoc:")
... print("pickup:", pickup_dt.isna().sum())
... print("dropoff:", dropoff_dt.isna().sum())
...
... print("\nThoi diem don khach sau khi chuyen:")
... print(pickup_dt.head())
...
```

```
Kieu du lieu truoc khi chuyen:
pickup     str
dropoff    str
dtype: object

Kieu du lieu sau khi chuyen:
pickup: datetime64[us]
dropoff: datetime64[us]

So gia tri khong chuyen doi duoc:
pickup: 0
dropoff: 0

Thoi diem don khach sau khi chuyen:
0   2019-03-23 20:21:09
1   2019-03-04 16:11:55
2   2019-03-27 17:53:01
3   2019-03-10 01:23:59
4   2019-03-30 13:27:42
Name: pickup, dtype: datetime64[us]
```

*Hình 4. Chuyển hai cột thời điểm đón và trả khách sang kiểu ngày giờ bằng pd.to_datetime().*

**Bước 5. Tạo cột thời lượng và cột tính tổng**

Tạo thoi_luong_phut = ((dropoff_dt - pickup_dt).dt.total_seconds() / 60).round(2), tức làm tròn hai chữ số. Đồng thời tạo tong_phi_co_ban = fare + tip + tolls. Chuyến đầu có thời lượng 375 giây / 60 = 6,25 phút và tổng ba khoản tiền là 7,00 + 2,15 + 0,00 = 9,15 USD. Cột tổng mới chỉ cộng ba khoản này, không thay thế cột total có sẵn của nguồn.

```
>>> df["thoi_luong_phut"] = (
...     (dropoff_dt - pickup_dt).dt.total_seconds() / 60
... ).round(2)
...
... df["tong_phi_co_ban"] = (
...     df["fare"] + df["tip"] + df["tolls"]
... ).round(2)
...
... print(df[
...     ["fare", "tip", "tolls", "tong_phi_co_ban", "thoi_luong_phut"]
... ].head().to_string(index=False))
...
... print("\nSo cot sau khi them:", df.shape[1])
...
 fare  tip  tolls  tong_phi_co_ban  thoi_luong_phut
  7.0 2.15    0.0             9.15             6.25
  5.0 0.00    0.0             5.00             7.08
  7.5 2.36    0.0             9.86             7.40
 27.0 6.15    0.0            33.15            25.87
  9.0 1.10    0.0            10.10             9.53

So cot sau khi them: 16
```

*Hình 5. Tạo cột thời lượng chuyến đi và cột tổng cước xe, tiền tip, phí cầu đường.*

**Bước 6. Chuẩn hóa cách trình bày ngày**

Sau khi tính thời lượng, dùng dt.strftime("%Y-%m-%d") để đưa pickup và dropoff về YYYY-MM-DD. Ví dụ 2019-03-23 20:21:09 được trình bày thành 2019-03-23. Tệp gốc vẫn giữ phần giờ để đối chiếu.

```
>>> print("Truoc khi chuan hoa:")
... print(df[["pickup", "dropoff"]].head().to_string(index=False))
...
... df["pickup"] = pickup_dt.dt.strftime("%Y-%m-%d")
... df["dropoff"] = dropoff_dt.dt.strftime("%Y-%m-%d")
...
... print("\nSau khi chuan hoa:")
... print(df[["pickup", "dropoff"]].head().to_string(index=False))
...
```

```
Truoc khi chuan hoa:
             pickup             dropoff
2019-03-23 20:21:09 2019-03-23 20:27:24
2019-03-04 16:11:55 2019-03-04 16:19:00
2019-03-27 17:53:01 2019-03-27 18:00:25
2019-03-10 01:23:59 2019-03-10 01:49:51
2019-03-30 13:27:42 2019-03-30 13:37:14

Sau khi chuan hoa:
    pickup    dropoff
2019-03-23 2019-03-23
2019-03-04 2019-03-04
2019-03-27 2019-03-27
2019-03-10 2019-03-10
2019-03-30 2019-03-30
```

*Hình 6. Chuẩn hóa hai cột ngày đón và trả khách về định dạng YYYY-MM-DD.*

**Bước 7. Xuất và kiểm tra CSV**

Dùng to_csv(index=False, encoding="utf-8-sig") để ghi ket_qua/taxis_da_lam_sach.csv. Đọc lại tệp xuất để kiểm tra: 6.341 dòng và 16 cột, gồm 14 cột gốc và hai cột mới. Không ghi đè dữ liệu gốc.

```
>>> from pathlib import Path
...
... Path("ket_qua").mkdir(exist_ok=True)
...
... df.to_csv(
...     "ket_qua/taxis_da_lam_sach.csv",
...     index=False,
...     encoding="utf-8-sig"
... )
...
... kiem_tra = pd.read_csv("ket_qua/taxis_da_lam_sach.csv")
...
... print("Da xuat:", Path("ket_qua/taxis_da_lam_sach.csv").resolve())
... print("So dong:", kiem_tra.shape[0])
... print("So cot:", kiem_tra.shape[1])
... print("Tong so o thieu:", kiem_tra.isna().sum().sum())
...
... print("\nDu lieu doc lai:")
... print(kiem_tra[
...     ["pickup", "dropoff", "thoi_luong_phut", "tong_phi_co_ban"]
... ].head().to_string(index=False))
...
```

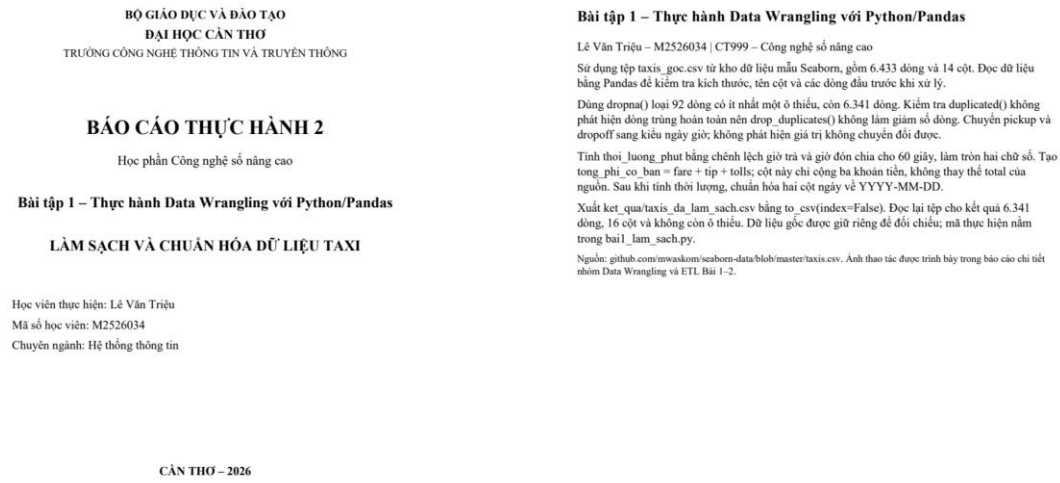
```
Da xuat: C:\lvtrieu\ths\CT999-Cong-nghe-so-nang-cao\bai-th-2\bai-1-2\ket_qua
\taxis_da_lam_sach.csv
So dong: 6341
So cot: 16
Tong so o thieu: 0

Du lieu doc lai:
    pickup    dropoff  thoi_luong_phut  tong_phi_co_ban
2019-03-23 2019-03-23             6.25             9.15
2019-03-04 2019-03-04             7.08             5.00
2019-03-27 2019-03-27             7.40             9.86
2019-03-10 2019-03-10            25.87            33.15
2019-03-30 2019-03-30             9.53            10.10
```

*Hình 7. Xuất CSV và đọc lại để kiểm tra số dòng, số cột, giá trị thiếu và kết quả tính toán.*

Kết quả Bài 1: đã xử lý dòng thiếu, kiểm tra trùng, chuẩn hóa ngày và tạo cột tính toán. Việc bỏ dòng thiếu trong bài này nhằm thực hành thao tác theo đề.

*Báo cáo ngắn được lập thành tài liệu riêng, gồm trang bìa và phần nội dung khoảng ½ trang mô tả các bước đã thực hiện. Hình dưới ghi nhận sản phẩm PDF.*



BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

**BÁO CÁO THỰC HÀNH 2**

Học phần Công nghệ số nâng cao

**Bài tập 1 – Thực hành Data Wrangling với Python/Pandas**

**LÀM SẠCH VÀ CHUẨN HÓA DỮ LIỆU TAXI**

Học viên thực hiện: Lê Văn Triệu
Mã số học viên: M2526034
Chuyên ngành: Hệ thống thông tin

**CẦN THƠ – 2026**

**Bài tập 1 – Thực hành Data Wrangling với Python/Pandas**

Lê Văn Triệu – M2526034 | CT999 – Công nghệ số nâng cao

Sử dụng tệp taxis_goc.csv từ kho dữ liệu mẫu Seaborn, gồm 6.433 dòng và 14 cột. Đọc dữ liệu bằng Pandas để kiểm tra kích thước, tên cột và các dòng đầu trước khi xử lý.

Dùng dropna() loại 92 dòng có ít nhất một ô thiếu, còn 6.341 dòng. Kiểm tra duplicated() không phát hiện dòng trùng hoàn toàn nên drop_duplicates() không làm giảm số dòng. Chuyển pickup và dropoff sang kiểu ngày giờ; không phát hiện giá trị không chuyển đổi được.

Tính thoi_luong_phut bằng chênh lệch giờ trả và giờ đón chia cho 60 giây, làm tròn hai chữ số. Tạo tong_phi_co_ban = fare + tip + tolls; cột này chỉ cộng ba khoản tiền, không thay thế total của nguồn. Sau khi tính thời lượng, chuẩn hóa hai cột ngày về YYYY-MM-DD.

Xuất ket_qua/taxis_da_lam_sach.csv bằng to_csv(index=False). Đọc lại tệp cho kết quả 6.341 dòng, 16 cột và không còn ô thiếu. Dữ liệu gốc được giữ riêng để đối chiếu; mã thực hiện nằm trong bai1_lam_sach.py.

Nguồn: github.com/mwaskom/seaborn-data/blob/master/taxis.csv. Ảnh thao tác được trình bày trong báo cáo chi tiết nhóm Data Wrangling và ETL Bài 1–2.

*Hình 7a. Trang bìa và trang nội dung của báo cáo Data Wrangling.*

Tệp PDF: m2526034-le-van-trieu/bao_cao_ngan/Bao_cao_Bai_1_Data_Wrangling_nua_trang.pdf. Bản Word cùng tên được lưu kèm.

*Minh chứng Bài 1: dữ liệu gốc du_lieu/taxis_goc.csv; mã nguồn bai1_lam_sach.py; dữ liệu sau xử lý ket_qua/taxis_da_lam_sach.csv. Các bước thực hiện và kết quả thể hiện tại Hình 1–7.*

## Bài tập 2 – Xây dựng pipeline ETL đơn giản

**Mục tiêu và công cụ**
Dùng Python và Pandas để xây dựng pipeline ETL đọc dữ liệu taxi từ CSV, chuẩn hóa dữ liệu và ghi thành CSV mới. Pipeline gồm ba hàm extract(), transform() và load(), được gọi lần lượt trong tệp bai2_etl.py.

### 2.1. Thực hiện pipeline ETL bằng Python

#### 2.1.1. Extract – Đọc dữ liệu

```
>>> du_lieu = extract()
>>> print("So dong:", du_lieu.shape[0])
... print("So cot:", du_lieu.shape[1])
... print(du_lieu[["color", "payment", "pickup"]].head().to_string(index=Fa\
lse))
...
So dong: 6433
So cot: 14
 color     payment              pickup
yellow credit card 2019-03-23 20:21:09
yellow        cash 2019-03-04 16:11:55
yellow credit card 2019-03-27 17:53:01
yellow credit card 2019-03-10 01:23:59
yellow credit card 2019-03-30 13:27:42
>>>
```

*Hình 8. Kết quả đọc dữ liệu CSV gốc và xem ba cột cần chuẩn hóa.*

Gọi hàm extract() để đọc tệp taxis_goc.csv bằng Pandas. Dữ liệu đầu vào gồm 6.433 dòng và 14 cột. Sau đó, xem 5 dòng đầu của ba cột color, payment và pickup để chuẩn bị cho bước chuẩn hóa dữ liệu.

#### 2.1.2. Transform – Chuẩn hóa dữ liệu

```
>>> def transform(df):
...     df = df.dropna().drop_duplicates().copy()
...     df["color"] = df["color"].str.strip().str.lower()
...     df["payment"] = df["payment"].str.strip().str.lower()
...     df["pickup"] = pd.to_datetime(df["pickup"], errors="raise").dt.strf\
time("%Y-%m-%d")
...     return df
...
```

```
...
>>> print("TRUOC CHUAN HOA:")
... print(du_lieu[["color", "payment", "pickup"]].head().to_string(index=Fa\
lse))
...
... du_lieu_sach = transform(du_lieu)
...
... print("\nSAU CHUAN HOA:")
... print(du_lieu_sach[["color", "payment", "pickup"]].head().to_string(ind\
ex=False))
... print("So dong:", du_lieu_sach.shape[0])
... print("So cot:", du_lieu_sach.shape[1])
... print("So o thieu:", du_lieu_sach.isna().sum().sum())
...
TRUOC CHUAN HOA:
 color     payment              pickup
yellow credit card 2019-03-23 20:21:09
yellow        cash 2019-03-04 16:11:55
yellow credit card 2019-03-27 17:53:01
yellow credit card 2019-03-10 01:23:59
yellow credit card 2019-03-30 13:27:42

SAU CHUAN HOA:
 color     payment     pickup
yellow credit card 2019-03-23
yellow        cash 2019-03-04
yellow credit card 2019-03-27
yellow credit card 2019-03-10
yellow credit card 2019-03-30
So dong: 6341
```

*Hình 9. Kết quả chuẩn hóa ba cột color, payment và pickup.*

Xây dựng hàm transform() để loại các dòng có dữ liệu thiếu và dòng trùng hoàn toàn. Sau đó, chuẩn hóa hai cột color, payment bằng cách bỏ khoảng trắng thừa và chuyển thành chữ thường; chuyển cột pickup về định dạng YYYY-MM-DD. Sau xử lý, dữ liệu còn 6.341 dòng, 14 cột và không còn ô thiếu.

### 2.1.3. Load – Lưu dữ liệu

```
>>> def load(df):
...     os.makedirs("ket_qua", exist_ok=True)
...     df.to_csv("ket_qua/taxis_etl.csv", index=False, encoding="utf-8-sig\
")
...
```

```
>>> load(du_lieu_sach)
...
... duong_dan = os.path.abspath("ket_qua/taxis_etl.csv")
... kiem_tra = pd.read_csv(duong_dan)
...
... print("LOAD - LUU VA KIEM TRA CSV")
... print("Tep da luu:", duong_dan)
... print("So dong:", kiem_tra.shape[0])
... print("So cot:", kiem_tra.shape[1])
... print("So o thieu:", kiem_tra.isna().sum().sum())
... print(kiem_tra[["color", "payment", "pickup"]].head().to_string(index=F\
alse))
...
LOAD - LUU VA KIEM TRA CSV
Tep da luu: C:\lvtrieu\ths\CT999-Cong-nghe-so-nang-cao\bai-th-2\m2526034-le-
van-trieu\ket_qua\taxis_etl.csv
So dong: 6341
So cot: 14
So o thieu: 0
 color     payment     pickup
yellow credit card 2019-03-23
yellow        cash 2019-03-04
yellow credit card 2019-03-27
yellow credit card 2019-03-10
yellow credit card 2019-03-30
>>>
```

*Hình 10. Lưu dữ liệu sau chuẩn hóa và đọc lại tệp CSV để kiểm tra.*

Gọi hàm load() để lưu dữ liệu sau chuẩn hóa vào tệp ket_qua/taxis_etl.csv. Tham số index=False giúp không ghi thêm cột chỉ số dòng. Sau đó, đọc lại tệp bằng read_csv() để kiểm tra. Kết quả gồm 6.341 dòng, 14 cột, không còn ô thiếu và cột pickup có định dạng YYYY-MM-DD.

## 2.2. Viết báo cáo

*Báo cáo được lập thành tài liệu riêng, gồm trang bìa và một trang nội dung giải thích pipeline ETL, lý do chọn Python/Pandas thay vì Excel/Google Sheets.*



BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO THỰC HÀNH 2

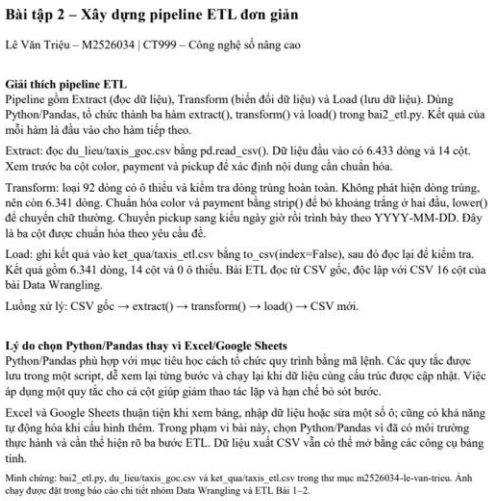
Học phần Công nghệ số nâng cao

Bài tập 2 – Xây dựng pipeline ETL đơn giản

ĐỌC, CHUẨN HÓA VÀ LƯU DỮ LIỆU TAXI

Học viên thực hiện: Lê Văn Triệu
Mã số học viên: M2526034
Chuyên ngành: Hệ thống thông tin

CẦN THƠ – 2026



Bài tập 2 – Xây dựng pipeline ETL đơn giản

Lê Văn Triệu – M2526034 | CT999 – Công nghệ số nâng cao

**Giải thích pipeline ETL**
Pipeline gồm Extract (đọc dữ liệu), Transform (biến đổi dữ liệu) và Load (lưu dữ liệu). Dùng Python/Pandas, tổ chức thành ba hàm extract(), transform() và load() trong bai2_etl.py. Kết quả của mỗi hàm là đầu vào cho hàm tiếp theo.

Extract: đọc du_lieu/taxis_goc.csv bằng pd.read_csv(). Dữ liệu đầu vào có 6.433 dòng và 14 cột. Xem trước ba cột color, payment và pickup để xác định nội dung cần chuẩn hóa.

Transform: loại 92 dòng có ô thiếu và kiểm tra dòng trùng hoàn toàn. Không phát hiện dòng trùng, nên còn 6.341 dòng. Chuẩn hóa color và payment bằng strip() để bỏ khoảng trắng ở hai đầu, lower() để chuyển chữ thường. Chuyển pickup sang kiểu ngày giờ rồi trình bày theo YYYY-MM-DD. Đây là ba cột được chuẩn hóa theo yêu cầu đề.

Load: ghi kết quả vào ket_qua/taxis_etl.csv bằng to_csv(index=False), sau đó đọc lại để kiểm tra. Kết quả gồm 6.341 dòng, 14 cột và 0 ô thiếu. Bài ETL đọc từ CSV gốc, độc lập với CSV 16 cột của bài Data Wrangling.

Luồng xử lý: CSV gốc → extract() → transform() → load() → CSV mới.

**Lý do chọn Python/Pandas thay vì Excel/Google Sheets**
Python/Pandas phù hợp với mục tiêu học cách tổ chức quy trình bằng mã lệnh. Các quy tắc được lưu trong một script, dễ xem lại từng bước và chạy lại khi dữ liệu cùng cấu trúc được cập nhật. Việc áp dụng một quy tắc cho cả cột giúp giảm thao tác lặp và hạn chế bỏ sót bước.

Excel và Google Sheets thuận tiện khi xem bảng, nhập dữ liệu hoặc sửa một số ô; cũng có khả năng tự động hóa khi cấu hình thêm. Trong phạm vi bài này, chọn Python/Pandas vì đã có môi trường thực hành và cần thể hiện rõ ba bước ETL. Dữ liệu xuất CSV vẫn có thể mở bằng các công cụ bảng tính.

Minh chứng: bai2_etl.py, du_lieu/taxis_goc.csv và ket_qua/taxis_etl.csv trong thư mục m2526034-le-van-trieu. Ảnh chạy được đặt trong báo cáo chi tiết nhóm Data Wrangling và ETL Bài 1–2.

*Hình 10a. Trang bìa và trang nội dung của báo cáo pipeline ETL.*

Tệp PDF: m2526034-le-van-trieu/bao_cao_ngan/Bao_cao_Bai_2_Pipeline_ETL_1_trang.pdf. Bản Word cùng tên được lưu kèm.

### 2.2.1. Giải thích pipeline ETL

Pipeline ETL là quy trình gồm ba bước: Extract (đọc dữ liệu), Transform (biến đổi dữ liệu) và Load (lưu dữ liệu). Trong bài thực hành, sử dụng Python và thư viện Pandas để đọc dữ liệu taxi từ tệp CSV, chuẩn hóa dữ liệu và lưu thành một tệp CSV mới.

Ở bước Extract, hàm extract() sử dụng pd.read_csv() để đọc tệp du_lieu/taxis_goc.csv. Dữ liệu ban đầu gồm 6.433 dòng và 14 cột. Xem trước ba cột color, payment và pickup để xác định nội dung cần chuẩn hóa.

Ở bước **Transform**, hàm transform() loại các dòng có ít nhất một ô thiếu và kiểm tra loại dòng trùng hoàn toàn. Bộ dữ liệu có 92 dòng chứa ô thiếu và không có dòng trùng hoàn toàn, nên sau xử lý còn 6.341 dòng. Tiếp theo, hai cột color và payment được bỏ khoảng trắng thừa ở hai đầu và chuyển thành chữ thường. Cột pickup được chuyển từ dạng ngày giờ sang định dạng YYYY-MM-DD.

Ở bước Load, hàm load() sử dụng to_csv() để lưu kết quả vào tệp ket_qua/taxis_etl.csv. Sử dụng index=False để không ghi thêm cột chỉ số dòng. Sau đó, đọc lại tệp để kiểm tra và thu được kết quả gồm 6.341 dòng, 14 cột, không còn ô thiếu.

Luồng xử lý của bài thực hành là:

**CSV gốc → extract() → transform() → load() → CSV kết quả.**

Ba hàm được gọi lần lượt, trong đó kết quả của bước trước là đầu vào cho bước sau. Khi cần xử lý lại dữ liệu có cùng cấu trúc, có thể chạy lại chương trình để thực hiện toàn bộ quy trình.

### 2.2.2. Lý do chọn Python thay cho Excel/Google Sheets:

Chọn Python kết hợp với Pandas vì công cụ này phù hợp với yêu cầu xây dựng một pipeline ETL bằng mã lệnh. Các thao tác đọc, chuẩn hóa và lưu dữ liệu được viết thành những hàm riêng, giúp dễ theo dõi và hiểu vai trò của từng bước.

Python cũng cho phép áp dụng cùng một quy tắc cho toàn bộ cột dữ liệu. Khi tệp đầu vào được cập nhật, có thể chạy lại chương trình để xử lý, giúp giảm thao tác thủ công và hạn chế việc bỏ sót bước. Mã nguồn được lưu trong tệp bai2_etl.py nên thuận tiện cho việc kiểm tra, chỉnh sửa và sử dụng lại.

Excel và Google Sheets thuận tiện khi cần xem bảng, nhập dữ liệu hoặc chỉnh sửa trực tiếp một số ô. Tuy nhiên, với bài thực hành này, chọn Python để học cách tổ chức quy trình xử lý tự động và thực hiện rõ ba bước Extract, Transform và Load.

*Minh chứng pipeline ETL: mã nguồn bai2_etl.py; đầu vào du_lieu/taxis_goc.csv; đầu ra ket_qua/taxis_etl.csv. Hình 8–10 ghi nhận ba bước Extract, Transform và Load.*